Circular Singly Linked List

What is a Circular Singly Linked List?

A **Circular Singly Linked List (CSLL)** is a linear data structure in which elements are stored in nodes, and each node contains a pointer to the next node in the sequence.

Unlike a regular singly linked list, the last node in a circular singly linked list does **not point to NULL**. Instead, it points back to the **first node (head)**, forming a circular structure.

Thus, the list forms a loop, and traversal can continue indefinitely.

Each node contains two parts:

1. **Data field** – stores the actual value
2. **Next field (Pointer)** – stores the address of the next node

The address of the first node is stored in the **head pointer**, and the last node points back to the **head**.

Structure of a Node

A node in a circular singly linked list consists of two components:

- **Data** – the value stored in the node
- **Next Pointer** – reference to the next node

Representation:

```
[ Data | Next ]
```

Example Circular Linked List:

```
Head → [10 | • ] → [20 | • ] → [30 | • ]
            ↑_____________________________|
```

Declaration of Circular Singly Linked List Node in C

The node structure is declared using a structure in C.

```
struct Node {
    int data;
    struct Node* next;
};
```

Explanation:

- `struct Node` defines the node type.
- `int data` stores the value of the node.
- `struct Node* next` stores the address of the next node.

To create a pointer to the head node:

```
struct Node* head;
```

Here, `head` points to the first node of the circular linked list.

Characteristics of Circular Singly Linked List

- Nodes are dynamically allocated in memory.
- Each node stores the address of the next node.
- The last node points back to the head node instead of NULL.
- Traversal can start from any node.
- The list forms a closed loop (circular structure).

Advantages of Circular Singly Linked List

1. Continuous Traversal

Since there is no NULL, traversal can continue from any node.

2. Efficient Use in Repetitive Tasks

Ideal for applications that require cyclic repetition.

3. No Need to Handle NULL

Simplifies some operations since there is no end NULL condition.

4. Efficient Insertion at End

With a tail pointer, insertion at the end can be done efficiently.

5. Useful in Circular Applications

Best suited for systems where processes repeat in a loop.

Disadvantages of Circular Singly Linked List

1. Complex Traversal

Traversal must be carefully controlled to avoid infinite loops.

2. No Direct End Detection

There is no NULL to indicate the end of the list.

3. Difficult Debugging

Errors in pointer handling can cause infinite loops.

4. Extra Care in Operations

Insertion and deletion operations are slightly more complex than SLL.

Real-Life Use Cases of Circular Singly Linked List

Circular linked lists are used in many real-world applications:

1. Round Robin Scheduling

Operating systems use circular lists to allocate CPU time to processes in a cyclic manner.

2. Multiplayer Games

Players take turns in a circular sequence.

3. Circular Buffer

Used in streaming data, keyboard buffers, and real-time systems.

4. Music Playlist Loop

Songs repeat continuously in loop mode.

5. Traffic Signal Systems

Signals change in a fixed cyclic order.

6. Resource Sharing Systems

Devices or tasks are accessed in a circular order.

Key Points to Remember

- A circular singly linked list connects the last node back to the head.
- There is no NULL pointer in the list.
- Each node contains data and a next pointer.
- Traversal can start from any node and continues in a loop.
- Extra care is needed to avoid infinite traversal.